

Scheduling algorithms for multi-stage flow shops with reworks under overlapped queue time limits

Hyeon-Il Kim & Dong-Ho Lee

To cite this article: Hyeon-Il Kim & Dong-Ho Lee (2023) Scheduling algorithms for multi-stage flow shops with reworks under overlapped queue time limits, International Journal of Production Research, 61:20, 6908-6922, DOI: [10.1080/00207543.2022.2139004](https://doi.org/10.1080/00207543.2022.2139004)

To link to this article: <https://doi.org/10.1080/00207543.2022.2139004>



Published online: 07 Nov 2022.



Submit your article to this journal [↗](#)



Article views: 1154



View related articles [↗](#)



View Crossmark data [↗](#)



Citing articles: 8 View citing articles [↗](#)

RESEARCH ARTICLE



Scheduling algorithms for multi-stage flow shops with reworks under overlapped queue time limits

Hyeon-Il Kim and Dong-Ho Lee 

Department of Industrial Engineering, Hanyang University, Seoul, Republic of Korea

ABSTRACT

This study addresses a multi-stage flow shop scheduling problem in which a job is reworked when its queue time between two arbitrary stages exceeds an upper limit. The problem is to determine the start times of jobs and rework setups/operations if incurred for the objective of minimising makespan. As an extension of the previous studies, multiple overlapped queue time limits are considered, i.e. some in-between stages of queue time limits for a job are overlapped. A mixed integer programming model is developed and its performance is reported for small-sized test instances. Then, due to the limited applications of optimal solution approaches, a variable neighbourhood search (VNS) algorithm is proposed that generates an initial solution and improves it using a shaking and a local search improvement methods. In addition, it is extended to the general variable neighbourhood search (GVNS) algorithms with variable neighbourhood descent (VND) methods. Computational results show that the GVNS algorithms outperform the VNS algorithm significantly and also give near optimal solutions for small-sized test instances within a reasonable amount of computation times.

ARTICLE HISTORY

Received 16 March 2022

Accepted 11 September 2022

KEYWORDS

Flow shop scheduling; overlapped queue time limits; reworks; mixed integer programme; variable neighbourhood search

1. Introduction

Flow shop scheduling is a well-known optimisation problem of determining the sequence or start times of the jobs to be processed on each serial stage. Due to theoretical challenges and practical importance, various types of the problem have been studied during the last decades. In the theoretical aspect, except for some special cases, it is an NP-hard problem that cannot be solved easily. Also, in the practical aspect, it has many industrial applications, especially high-volume and low-variety type manufacturing systems such as electronics assembly, semiconductor fabrication, steel-making, and so on.

The classical flow shop scheduling problem is evolved into various extensions with sequence-dependent setups, no-wait or queue time limits, energy consumption, and so on (Yenisey and Yagmahan 2014; Rossit, Tohmé, and Frutos 2018; Komaki, Sheikh, and Malakooti 2019). For examples, refer to Benkalai et al. (2017), Jiang and Wang (2019) and Ramezani, Vali-Siar, and Jalalian (2019) for the problems with sequence-dependent setups; Samarghandi and ElMekkawy (2012), Gao et al. (2018), Panwalker and Koulamas (2021), Schaller and Valente (2021) and Zhao et al. (2022) for no-wait flow shops;

and Wang and Wang (2019), Zheng et al. (2020), Ho, Hnaïen, and Dugardin (2021) and Xin et al. (2021) for the problems with energy consumption. Among them, the queue time limits occur in various manufacturing systems. For example, in wafer fabrication, after a wafer that has undergone a chemical treatment on a diffusion machine, the next operation for the wafer must be started on another diffusion machine within a time limit. In this case, when the queue time limit is violated, the wafer must be reworked or scrapped because the chemical treatment is no longer effective. For more details, see Bai and Gershwin (1990), Scholl and Dumaschke (2000), Zhou and Wu (2017) and Pirovano et al. (2020). Additionally, queue time limits can be found in other industries, e.g. Harjunkoski and Grossmann (2001) for a steel making process, Akkerman, van Donk, and Gaalman (2007) for a food production process and Gicquel et al. (2012) for a bio process.

In general, queue time limits can be classified into consecutive, continuous, non-overlapped and overlapped types. The simplest consecutive type is a single limit between two successive operations, e.g. QTL_{12} in Figure 1(a), and the continuous type is a single one

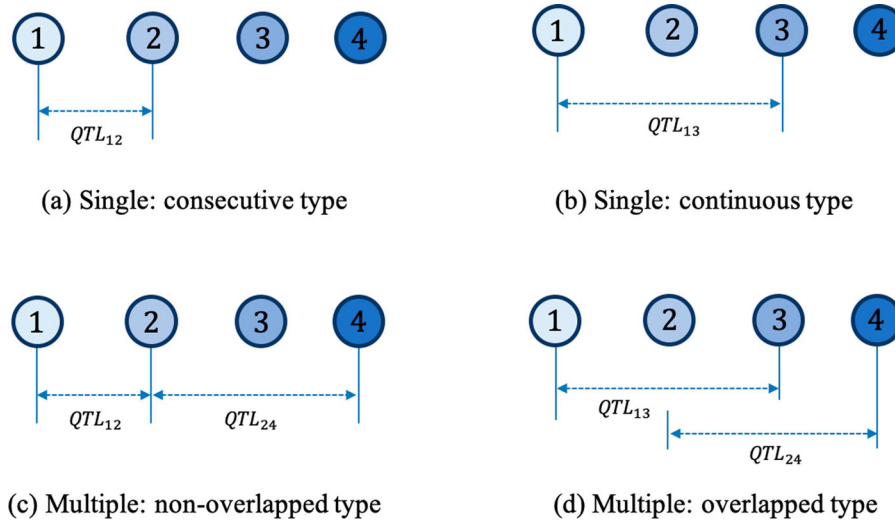


Figure 1. Types of queue time limits.

Table 1. Literatures on flow shop scheduling with queue time limits.

Types of queue time limits	Number of stages	Articles
Consecutive	Two	Yang and Chern (1995), Su (2003), Bouquard and Lenté (2006), Joo and Kim (2009), Ham, Lee, and An (2011), Wang, Li, and Ge (2013), An, Kim and Choi (2016), Shi and Gershwin (2016), Yu, Kim, and Lee (2017), Wang, Ju and Kang (2019), Wang et al. (2020), Li and Dai (2020), Lee (2020)
	Multiple	Chen and Yang (2006), Wang and Li (2011), Ham (2012)
Continuous	Multiple	Klemmt and Mönch (2012)
Non-overlapped	Multiple	Dhouib, Teghem, and Loukil (2013), Hamdi and Loukil (2015), Wang, Huang, and Li (2018)
Overlapped	Multiple	Kim and Lee (2019), Lee, Yu, and Park (2021)

between two arbitrary, not always consecutive, stages, e.g. QTL_{13} in Figure 1(b), where $QTL_{m,m'}$ denotes a queue time limit between stages m and m' . Also, the non-overlapped type is a set of multiple and non-overlapped limits, e.g. QTL_{12} and QTL_{24} in Figure 1(c), and the overlapped type is the most generalised one with multiple and overlapped limits, e.g. QTL_{13} and QTL_{24} in Figure 1(d).

The previous studies on flow shop scheduling with queue time limits can be classified by the types of queue time limits. See Table 1 for details.

As can be seen in Table 1, most studies are done for two-machine flow shops with the consecutive type. Yang and Chern (1995) consider the problem that minimises makespan and propose a branch and bound algorithm after proving the NP-hardness. Joo and Kim (2009) improve the branch and bound algorithm by

developing dominance properties and new lower bounds, and An, Kim and Choi (2016) propose another branch and bound algorithm for an extended problem with sequence-dependent setups. Ham, Lee, and An (2011) propose integer programming-based dispatching heuristics for a two-machine batching problem, and Yu, Kim, and Lee (2017) propose heuristics that minimise the total waiting time variation for a problem with operation skips, i.e. some jobs skip their operations on the first machine. Also, Li and Dai (2020) propose a hybrid membrane computing metaheuristic that minimises makespan for a batch machine followed by a discrete machine, where jobs in incompatible job families must be processed on the discrete machine within a limited queue time after completion of the batch process, and Lee (2020) proposes a genetic algorithm that minimises the total tardiness for a two-machine problem with sequence-dependent setups on the second machine. Su (2003), Bouquard and Lenté (2006), Wang, Li, and Ge (2013), Shi and Gershwin (2016), Wang, Ju and Kang (2019) and Wang et al. (2020) for other two-machine problems with the consecutive type.

The above studies are extended to multi-stage flow shops. For a single queue time limit, Chen and Yang (2006) develop two integer programming models for the makespan problem with the consecutive type, and Wang and Li (2011) report the performance of permutation heuristics. Ham (2012) proposes an integer programming-based real-time dispatching heuristic for an etching station in wafer fabrication. Also, Klemmt and Mönch (2012) develop a mixed integer programming model for the tardiness problem with the continuous type between two arbitrary stages and propose a list scheduling heuristic. For multiple non-overlapped

type, Dhouib, Teghem, and Loukil (2013) propose a simulated annealing algorithm that minimises the number of tardy jobs first and the makespan second for the problem with lower and upper queue time limits for every pair of consecutive operations, and later, Hamdi and Loukil (2015) propose a heuristic with a Lagrangian relaxation based lower bound for the tardiness problem. Also, Wang, Huang, and Li (2018) propose a two-stage constructive heuristic for the makespan problem with a queue time limit between consecutive operations. Recently, Kim and Lee (2019) proposed a branch and bound algorithm after deriving dominance properties for a three-machine problem with overlapped queue time limits, and Lee, Yu, and Park (2021) proposed a genetic algorithm.

This study focuses on multi-stage flow shop scheduling with reworks under multiple overlapped queue time limits, where a job must be reworked on the corresponding stages when a queue time limit is violated. For example, multiple overlapped queue time limits can be found between a cleaning process (C) and two successive deposition processes (D1 and D2), i.e. one between C and D1 and the other between C and D2, in display manufacturing systems. In this example, if one of the limits is violated, a rework occurs from the cleaning process after the deposited material is removed, i.e. rework setup. The problem is to determine the start times of jobs and rework setups/operations if incurred for the objective of minimising makespan. In fact, this study extends Kim and Lee (2019) and Lee, Yu, and Park (2021) in two aspects. First, a rework, commonly occurs in manufacturing, is considered. Note that in the previous studies, if a job violates a queue time limit, the resulting schedules are infeasible. Second, this study considers flow shops with more than three stages and hence is more applicable to practical systems. In summary, to the best of the authors' knowledge, there is no previous study on multi-stage flow shop scheduling with reworks under multiple overlapped queue time limits.

A mixed integer programming model is developed to represent the new problem mathematically. Due to the limited applications of the optimal solution approaches, a variable neighbourhood search algorithm is proposed that obtains an initial solution and improves it using a shaking and a local search improvement methods. Then, it is extended to general variable neighbourhood search algorithms with variable neighbourhood descent methods. To test the performance of the algorithms, computational experiments were done, and the results are reported.

The rest of this paper is organised as follows. Section 2 describes the problem in more detail and presents the mixed integer programming model. Sections 3

and 4 present the variable neighbourhood search algorithms and test results, respectively. Finally, Section 5 summarises this paper and proposes further research topics.

2. Problem description

The system considered in this study consists of M serial stages and 1 rework setup station, each of which has a single processor with a sufficient buffer capacity. There are n jobs to be processed by M operations with linear precedence relations. In particular, each job may have two or more overlapped queue time limits with possible reworks. In other words, if any of the overlapped queue time limits of a job is violated, the job must return to the stage where the queue time limit starts and the corresponding rework operations are done from the stage after a rework setup is performed on the rework setup station. Note that the queue time between stages m and m' is defined as the cumulative sum of waiting times at stages $m, m+1, \dots, m'$. Figure 2 shows an example of the system with two overlapped queue time limits, i.e. one between stages m and m' and the other between $m+1$ and M . In this case, if the queue time limit between stages m and m' is violated while a job is waiting at stage r ($r \leq m'$), the job must return to stage m after the rework setup is performed on the rework setup station and then the corresponding rework operations are done from stage m to $r-1$.

For the system described above, the problem is to determine: (a) the start times of the jobs on each stage; and (b) the start times of rework setups/operations on the rework setup station and corresponding stages if incurred. The objective is to minimise makespan. As a beginning study on the new flow shop scheduling problem, this study considers a static and deterministic version of the problem. In other words, all jobs are available at the beginning, i.e. zero ready times, and processing times, rework setup times and queue time limits are deterministic and given in advance. Other assumptions made for the problem are: (a) each machine can process only one job at a time; (b) setup times are sequence-independent; (c) transportation times are negligible; (d) machine breakdowns are not considered; and (e) rework can be done once for each job due to quality problems.

The problem can be formulated using the process route-based model of Cho et al. (2019). Before presenting the model, two types of process routes, i.e. normal and abnormal ones, are explained first. The normal process route for a job is the sequence of operations to be performed on the corresponding stages, i.e. operation m on the m th stage, and the abnormal process routes

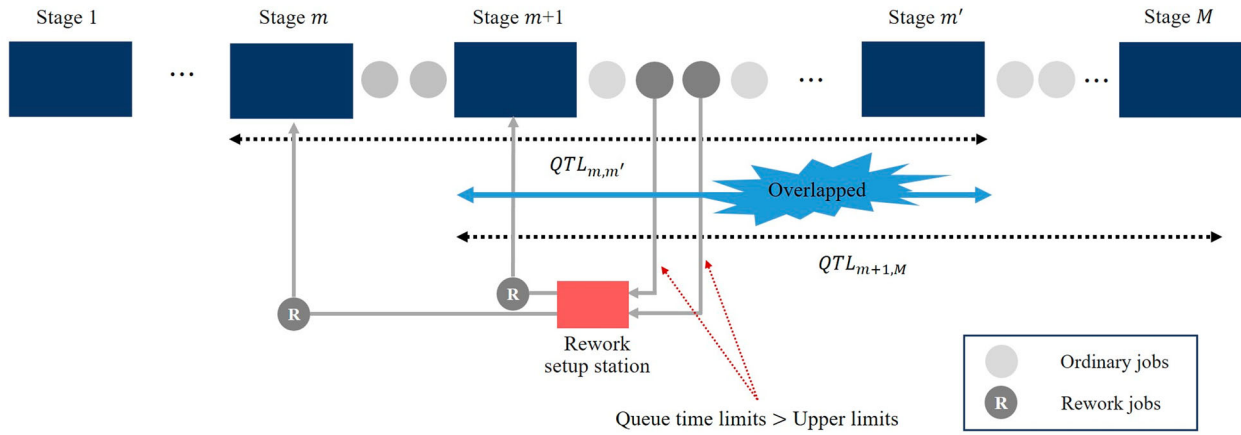


Figure 2. System configuration: example.

are the ones with reworks due to queue time violations, i.e. processing, rework setup and rework operations. The two types of process routes can be represented as a network with source, intermediate and sink nodes, where the source and sink (s and t) represent the start and end of a job and the intermediate nodes represent processing (1, 2, ..., M), rework setup (RS) and rework operations (R1, R2, ..., RM). Also, the arcs represent the precedence relations between consecutive operations. Figure 3 illustrates a network for a job with two overlapped queue time limits, i.e. one between stages 1 and 3 and the other between stages 2 and 4, in four-stage flow shops. In the figure, the normal route is $s - 1 - 2 - 3 - 4 - t$, and the abnormal routes are $s - 1 - RS - R1 - 2 - 3 - 4 - t$ and $s - 1 - 2 - RS - R1 - R2 - 3 - 4 - t$ when the time limit between stages 1 and 3 is violated at stages 2 and 3, respectively. Similarly, when the time limit between stages 2 and 4 is violated at stages 3 and 4, the abnormal routes are $s - 1 - 2 - RS - R2 - 3 - 4 - t$ and $s - 1 - 2 - 3 - RS - R2 - R3 - 3 - 4 - t$. In this way, the network can represent all possible process routes.

Now, the problem can be represented as the following mixed integer programming model. The notations used are summarised in Table 2.

[P] Minimise C_{max}
subject to

$$\sum_{p \in P_i} Z_{ip} = 1 \text{ for all } i \in I \quad (1)$$

$$X_{ipj} = Z_{ip} \text{ for all } i \in I, p \in P_i \text{ and } j \in O_{ip} \quad (2)$$

$$S_{ipj} + C_{ipj} \leq M \cdot X_{ipj} \text{ for all } i \in I, p \in P_i \text{ and } j \in O_{ip} \quad (3)$$

$$S_{ipj} + t_{ipj} \leq C_{ipj} + M \cdot (1 - X_{ipj}) \text{ for all } i \in I, p \in P_i \text{ and } j \in O_{ip} \quad (4)$$

$$S_{ipj} + M \cdot Y_{ipji'p'j'} \geq C_{i'p'j'} \text{ for all } i, i' \in I, p, p' \in P_i,$$

Table 2. Notation used in the mixed integer programming model.

Indices	
i	jobs, $i \in I$
p	process routes, $p \in P_i$, where $p = 1$ for normal route, and $p > 1$ for abnormal routes of job i
j	ordered operations in each of normal or abnormal process route of a job, $j \in O_{ip}$, where O_{ip} denotes the ordered set of operations in process route p of job i . (f_{ip} and l_{ip} denote the indices for the first and the last operations in process route p of job i , respectively.)
k, k'	operations in each process route of a job
Parameters	
t_{ipj}	processing time of operation j in process route p of job i
$Q_{ikk'}$	queue time limit between operations k and k' of job i ($k < k'$)
M	large number
Decision variables	
Z_{ip}	= 1 if process route p is selected for job i , and 0 otherwise
X_{ipj}	= 1 if job i is processed to operation j in process route p , and 0 otherwise
$Y_{ipji'p'j'}$	= 1 if operation j in process route p of job i precedes operation j' in process route p' of job i' , and 0 otherwise
S_{ipj}	start time of operation j in process route p of job i
C_{ipj}	completion time of operation j in process route p of job i
C_i	completion time of job i

$$j \in O_{ip} \text{ and } j' \in O_{i'p'} (i \neq i') \quad (5)$$

$$S_{i'p'j'} + M \cdot (1 - Y_{ipji'p'j'}) \geq C_{ipj} \text{ for all } i, i' \in I, p,$$

$$p' \in P_{i'}, j \in O_{ip} \text{ and } j' \in O_{i'p'} (i \neq i') \quad (6)$$

$$S_{ipj} \geq C_{i,p,j-1} \text{ for all } i \in I, p \in P_i \text{ and } j \in O_{ip}/\{f_{ip}\} \quad (7)$$

$$S_{i1k} - C_{i1k} \leq Q_{ikk'} \text{ for all } i \in I \text{ and } k, k' \in O_{ip} (k < k') \quad (8)$$

$$C_{max} \geq C_{i,p,l_{ip}} \text{ for all } i \in I \text{ and } p \in P_i \quad (9)$$

$$S_{ipj}, C_{ipj}, C_i \geq 0 \text{ for all } i \in I, p \in P_i \text{ and } j \in O_{ip} \quad (10)$$

$$Z_{ip}, X_{ipj}, Y_{ipji'p'j'} \in \{0, 1\} \text{ for all } i, i' \in I, p, p' \in P_i,$$

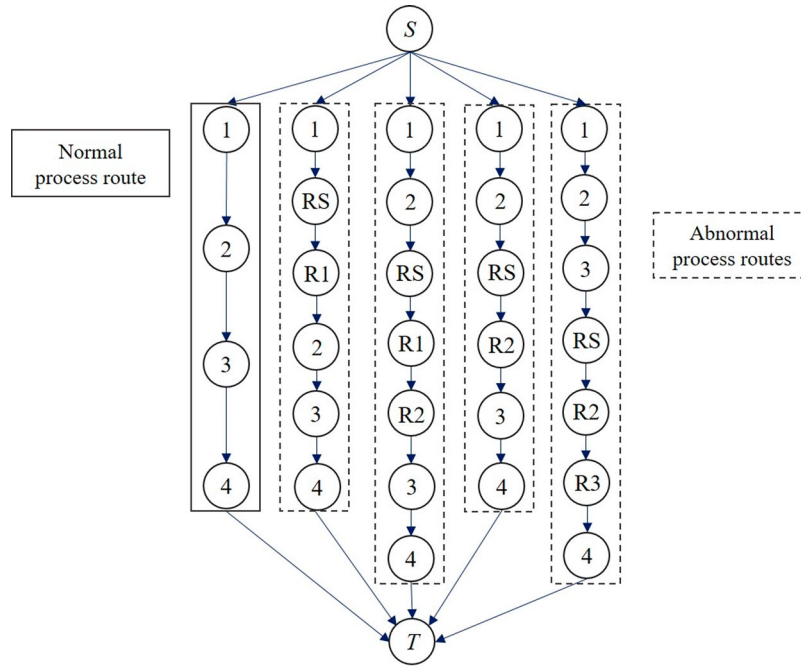


Figure 3. Network representation of process routes: example.

$$j \in O_{ip} \text{ and } j' \in O_{i'p'} \ (i \neq i') \quad (11)$$

The objective with constraint (9) represents makespan, i.e. maximum completion time. Constraint (1) ensures that only one process route must be selected for each job. Constraint (2) represents that the corresponding operations, including possible reworks, must be done when a process route is selected for a job, and constraints (3) and (4) specify the resulting start and completion times. Constraints (5) and (6), which are similar to the disjunctive constraints of the job shop scheduling problem, ensure that no two operations can be processed simultaneously on a stage. Specifically, constraint (6) works if the start time of operation j in process route p of job i is greater than or equal to the completion time of operation j' in process route p' of job i' , i.e. $Y_{ipji'p'j'} = 1$, while constraint (5) works, otherwise. Constraint (7) represents the precedence relations between consecutive operations of each job, and constraint (8) represents that the queue time limits must be satisfied for normal process routes. Recall that $p = 1$ for the normal process route of each job. Finally, the remaining constraints represent the conditions of decision variables.

Although the problem [P] can be solved using a commercial software package, it is impractical for large-sized instances due to excessive computation times. In fact, the special case without queue time limits is the ordinary multi-stage flow shop makespan scheduling problem that is proved to be NP-hard (Pinedo 2010). Therefore,

heuristic algorithms are proposed in this study because the optimal approach has very limited applications.

3. Solution algorithms

3.1. Variable neighbourhood search (VNS) algorithm

The VNS algorithm, proposed by Mladenović and Hansen (1997), is a meta-heuristic that explores a set of predefined neighbourhood structures successively within a local search algorithm for the purpose of enlarging the search space while escaping from local optimums. See Hansen and Mladenović (2001, 2003) and Hansen, Mladenović, and Urošević (2006, 2017) for the details on the VNS algorithm. The VNS algorithm is adopted in this study because of its successful applications to various flow shop scheduling problems (Samarghandi and ElMekkawy 2012; M'hallah 2014; Xie et al. 2014; Moslehi and Khorasanian 2014; Wu and Che 2020; Jomaa, Eddaly, and Jarboui 2021; Shao, Shao, and Pi 2022). Figure 4 shows the VNS algorithm proposed in this study, which consists of four steps: (a) obtaining an initial solution; (b) shaking that generates neighbourhoods and selects one of them; (c) local search that improves the selected solution by exploring its neighbours; and (d) checking the termination condition. As can be seen in the figure, the shaking and local search improvement steps are repeatedly done until the termination condition is satisfied.

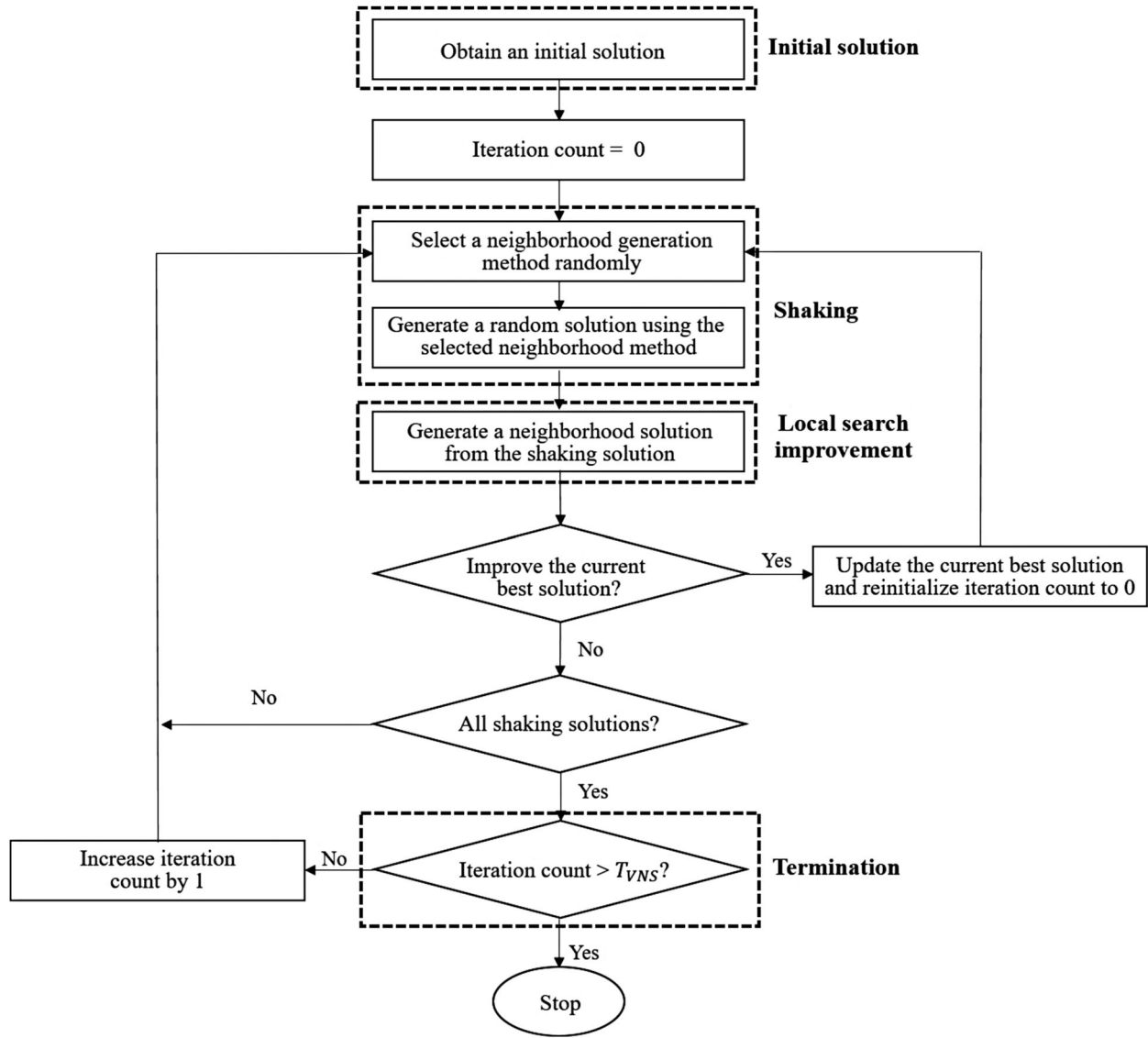


Figure 4. Variable neighbourhood search (VNS) algorithm.

3.1.1. Obtaining an initial solution

A solution is represented by a permutation $\pi = (\pi_1, \pi_2, \dots, \pi_n)$ of job sequence at the first stage, where π_j denotes the index of the job at j th position in the permutation. Because the permutation implies the job sequence at the first stage, the complete schedule including the job sequences at the remaining stages and rework setup station must be determined. For this purpose, a priority scheduling method is used that selects the best one among the non-delay schedules generated by priority rules because of the complexity of the resulting problem for the remaining stages and rework setup station. Therefore, the resulting complete schedule is non-permutation, i.e. non-identical job sequences on machines.

In this study, the following priority rules, which are usually used for job shop makespan scheduling, are

tested. Let t denotes the time at which a priority rule is applied.

FCFS (first come first served): select a job that arrived the earliest at the queue of the stage, i.e. select a job i^* such that

$$i^* = \operatorname{argmin}_{i \in N_{mt}} \{a_{im}\},$$

where N_{mt} and a_{im} denote the set of jobs waiting for processing in the queue of stage m at time t and the arrival time of job i at stage m , respectively

SOPT (shortest operation processing time): select a job with the shortest operation processing time, i.e. select a job i^* such that

$$i^* = \operatorname{argmin}_{i \in N_{mt}} \{p_{im}\},$$

where p_{im} denotes operation processing time of job i on stage m

MWKR (most work remaining): select a job with the largest remaining work, i.e. select a job i^* such that

$$i^* = \operatorname{argmax}_{i \in N_{mt}} \{r_{im}\},$$

where r_{im} remaining work of job i on stage m , i.e. sum of processing times of successor operations including itself

PWR (processing time over remaining work): select a job with the smallest ratio of operation processing time to remaining work, i.e. select a job i^* such that

$$i^* = \operatorname{argmin}_{i \in N_{mt}} \{p_{im}/r_{im}\}$$

An initial solution $\mathbf{x}$ is obtained by an NEH-based heuristic. The NEH heuristic, developed by Nawaz, Ensco, and Ham (1983), is a well-known permutation flow shop scheduling heuristic that expands a partial job sequence gradually by inserting a job into the best position of the partial sequence at a time. Note that the complete schedule for a given partial or full job sequence is constructed using the priority scheduling method explained above. The detailed procedure is given below.

Procedure. (Obtaining an initial solution)

Step 1. Sort the jobs in the non-increasing order of the total processing times, i.e. sums of processing times over all stages.

Step 2. Select the better one of the two sequences for the first two jobs at the first stage, where the complete schedule is obtained using the priority scheduling method.

Step 3. Insert the next job in the sorted list to the best position that gives the minimum makespan. Here, a tie is broken by selecting the position that gives the smallest completion time on the next stage. Do this repeatedly until all jobs are scheduled.

3.1.2. Shaking

Shaking is done to resolve any local optimum traps by changing neighbourhood solutions in a systematic way. The shaking method proposed in this study consists of selecting a neighbourhood generation method and generating a random solution using the selected neighbourhood method, i.e. generate a random solution $\mathbf{x}' \in N'(\mathbf{x})$ for the current solution $\mathbf{x}$, where $N'(\mathbf{x})$ denotes the set of solutions generated by the selected neighbourhood method. Note that the random selection is done to avoid cycling that may occur if a deterministic method is used, called the random jump in the literature. In this study, the following neighbourhood generation methods are proposed.

Swap. The swap method generates neighbourhood solutions by interchanging the positions of two jobs located in the current solution. Specifically, the string of the current solution is divided in half, i.e. former S_1

and latter S_2 . Then, two jobs J_i and $J_{i'}$ such that $J_i \in S_1$ and $J_{i'} \in S_2$ are selected randomly and they are interchanged. See Figure 5(a) for an example.

Insertion. After selecting two jobs randomly, the insertion method generates a neighbourhood solution by removing the first job from the original place and inserting it to the position that directly precedes the second job. Specifically, the string of the current solution is divided in half, i.e. S_1 and S_2 . Then, two jobs J_i and $J_{i'}$ such that $J_i \in S_1$ and $J_{i'} \in S_2$ are selected randomly and the first job J_i is removed from its position and inserted to the position that directly precedes the second job $J_{i'}$. See Figure 5(b) for an example.

Reverse. The reverse method selects two jobs randomly and reverses the order of the jobs between the two jobs. As in the other methods, after dividing the string of the current solution in half, i.e. S_1 and S_2 , two jobs J_i and $J_{i'}$ such that $J_i \in S_1$ and $J_{i'} \in S_2$ are selected randomly and the order of the jobs between the two jobs J_i and $J_{i'}$ are reversed. See Figure 5(c) for an example.

3.1.3. Local search improvement

This step improves the shaking solution $\mathbf{x}'$ further using the swap, insertion and reverse methods explained earlier. Specifically, the three neighbourhood generation methods are applied to $\mathbf{x}'$ and the best one $\mathbf{x}''$ is selected. Then, if $\mathbf{x}''$ improves the current solution $\mathbf{x}$, the current solution is updated and the next shaking is done. Otherwise, the local search improvement is done for another shaking solution obtained using the neighbourhood methods not used in the current shaking until all the neighbourhood methods are used. Here, the iteration count is reinitialized to 0 if an improved solution is obtained, and increased by 1, otherwise. The detailed procedure is given below.

Procedure. (Local search improvement of the VNS algorithm)

Step 1. Generate neighbourhood solutions using the swap, insertion and reverse methods for the shaking solution and select the best one.

Step 2. If the best solution improves the current solution, update the current solution and stop the local search improvement after reinitializing the iteration count to 0. Otherwise, go to step 3.

Step 3. If all the neighbourhood methods are used for the current shaking, stop the local search improvement after increasing the iteration count by 1. Otherwise, obtain another best solution using the neighbourhood methods not used in the current shaking and go to Step 2.

3.1.4. Termination

For a given initial solution, the shaking and local search improvement steps are repeatedly done until there is no

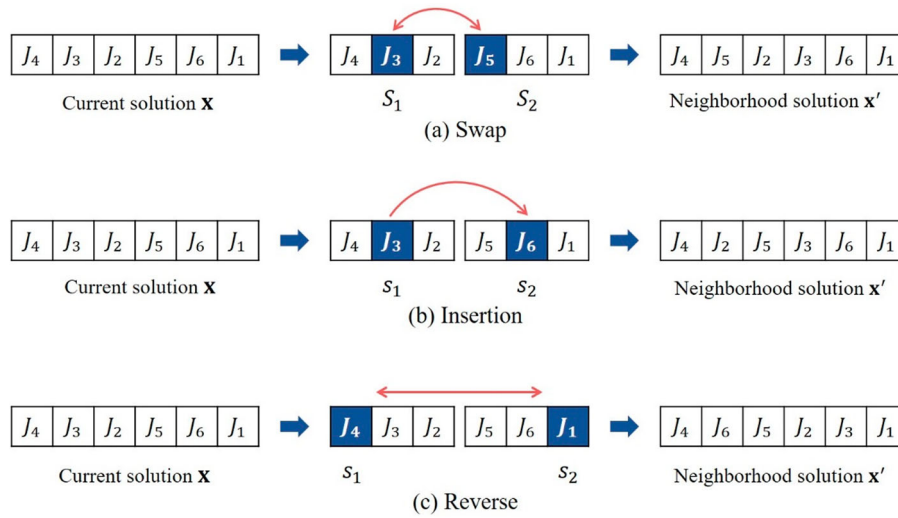


Figure 5. Neighbourhood generation methods: examples.

improvement for a certain number T_{VNS} of consecutive iterations without improvement.

3.2. General variable neighbourhood search (GVNS) algorithm

The GVNS algorithm, developed by Hansen, Mladenović, and Urošević (2006), enlarges the search space of the VNS algorithm by the variable neighbourhood decent (VND) algorithm that uses different neighbourhood generation methods repeatedly in the shaking and local search improvement steps. Figure 6 shows an overview of the GVNS algorithm proposed in this study.

As can be seen in the figure, the GVNS algorithm generates a shaking solution using the current neighbourhood generation method according to a given order of the neighbourhood generation methods, i.e. VND algorithm. Then, a neighbourhood solution is generated from the shaking solution using the current neighbourhood generation method of the VND algorithm in the local search improvement. If an improved solution is obtained, the current best solution is updated and the iteration count is reinitialized to 0. Otherwise, the next neighbourhood solution is generated from the shaking solution using the next neighbourhood method of the VND algorithm in the local search improvement. If no improved solution can be obtained, the next shaking solution is generated using the VND algorithm. Note that the iteration count is increased by 1 when no improved solution can be obtained after the shaking and local search improvement steps. As in the VNS algorithm, the shaking and local search improvement steps are repeatedly done until there is no improvement for T_{GVNS} consecutive iterations without improvement.

For a given order of neighbourhood methods in each of shaking and local search improvement, the detailed procedure is given below.

Procedure. (Shaking and local search improvement of the GVNS algorithm)

Step 1. Obtain a shaking solution by generating a random solution using the first shaking neighbourhood method.

Step 2. For the current shaking solution, generate a random neighbourhood solution using the current neighbourhood generation method of local search improvement. If an improved solution is obtained, update the current best solution and reinitialize the iteration count to 0. Otherwise, go to Step 3.

Step 3. If all neighbourhood methods in local search improvement are considered, go to Step 4. Otherwise, set the current neighbourhood generation method to the next one and go to Step 2.

Step 4. If all shaking solutions are considered, increase the iteration count by 1. If the iteration count is greater than T_{GVNS} , stop. Otherwise, generate a shaking solution using the next shaking neighbourhood method and go to Step 2.

4. Computational results

To test the performance of the algorithms proposed in this study, computational experiments were done and the results are reported in this section. The algorithms were coded in C++ and the tests were done on a PC with an AMD Ryzen 5 Pro 4650G CPU at 3.70 GHz clock speed with 16GB RAM memory.

Before the main tests, a preliminary test was done to find the best combination of neighbourhood orders in the GVNS algorithm. Note that there exist 3! orders that use

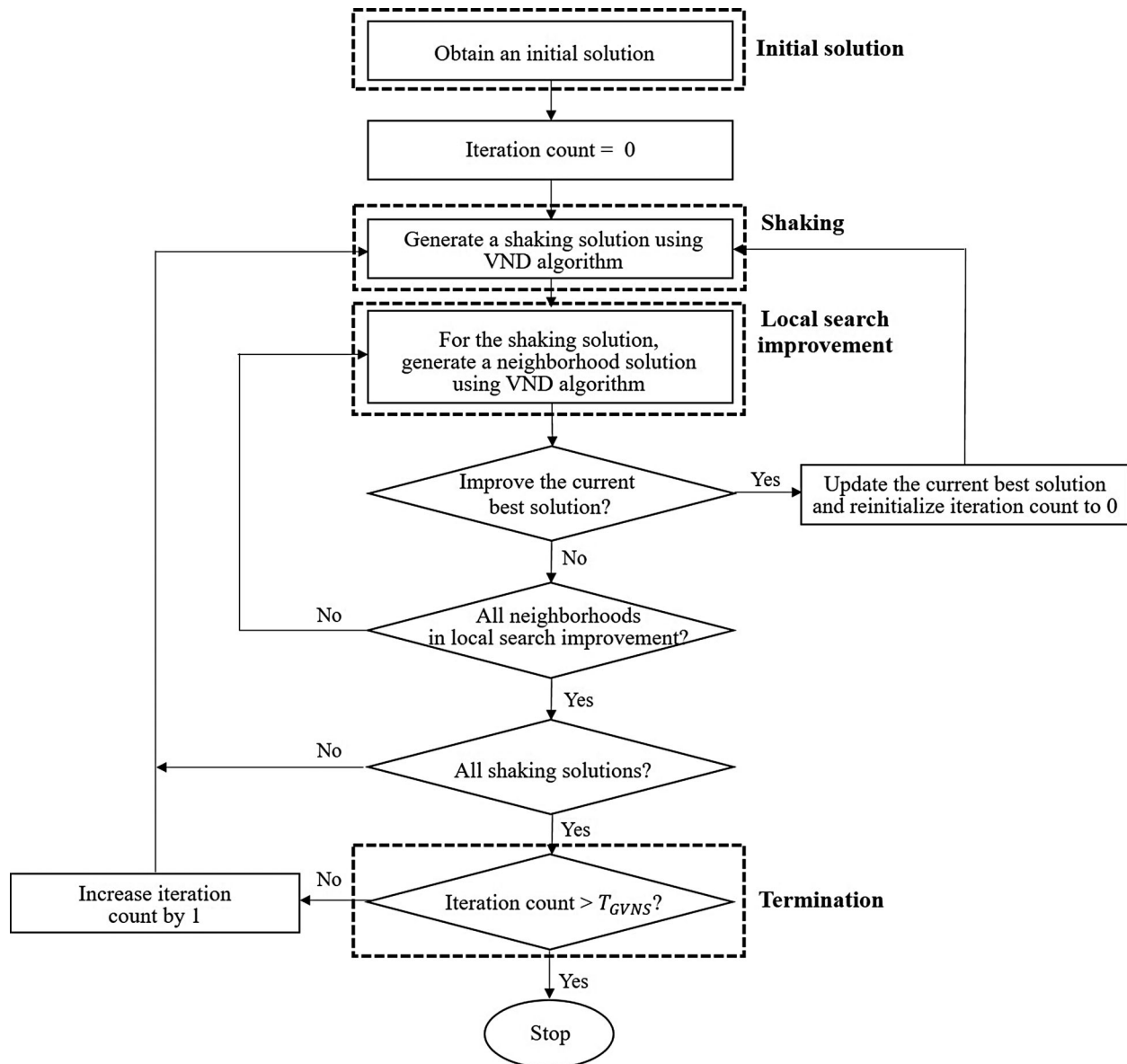


Figure 6. General variable neighbourhood search (GVNS) algorithm.

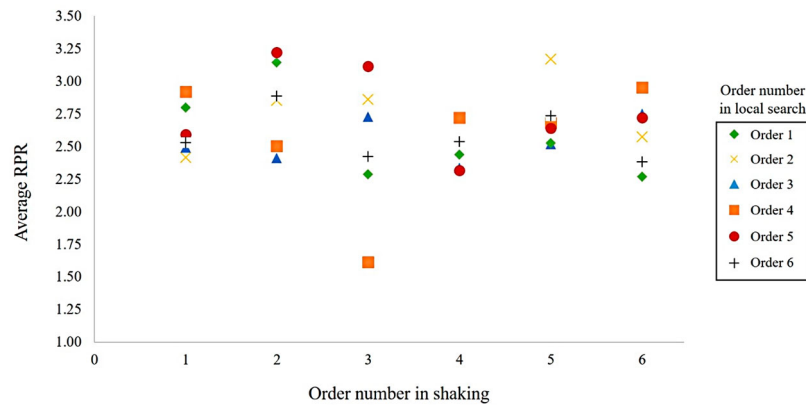


Figure 7. Results for the preliminary test on GVNS algorithms.

Table 3. Result of Duncan's multiple range test on GVNSs with different order combinations.

Order combinations (<i>a</i> – <i>b</i>)*	Clusters	
Order 3 – Order 4	1	
Order 6 – Order 1	1	2
Order 3 – Order 1	1	2
Order 4 – Order 5		2 3
Order 4 – Order 3		2 3
Order 4 – Order 2		2 3
Order 6 – Order 6		2 3
Order 3 – Order 6		2 3
Order 4 – Order 1		2 3
Order 2 – Order 3		2 3
Order 1 – Order 3		2 3
Order 2 – Order 4		3 4
Order 5 – Order 1		3 4
Order 1 – Order 2		3 4
Order 5 – Order 3		3 4
Order 1 – Order 6		3 4
Order 4 – Order 6		3 4
Order 6 – Order 2		3 4
Order 1 – Order 5		3 4
Order 5 – Order 5		3 4
Order 5 – Order 4		3 4
Order 6 – Order 5		3 4
Order 4 – Order 4		3 4
Order 3 – Order 3		3 4
Order 5 – Order 6		3 4
Order 6 – Order 3		3 4
Order 1 – Order 1		3 4
Order 2 – Order 2		3 4
Order 3 – Order 2		3 4
Order 2 – Order 6		3 4
Order 1 – Order 4		3 4
Order 6 – Order 4		3 4
Order 3 – Order 5		3 4
Order 2 – Order 1		3 4
Order 5 – Order 2		3 4
Order 2 – Order 5		5

*Orders of shaking (*a*) and local search improvement (*b*).

Order 1: swap – insertion – reverse.
 Order 2: swap – reverse – insertion.
 Order 3: insertion – swap – reverse.
 Order 4: insertion – reverse – swap.
 Order 5: reverse – insertion – swap.
 Order 6: reverse – swap – insertion.

the three neighbourhood methods in each of the shaking and the local search improvement steps and hence 36 order combinations in total, i.e. order 1 (swap – insertion – reverse), order 2 (swap – reverse – insertion), order 3 (insertion – swap – reverse), order 4 (insertion – reverse – swap), order 5 (reverse – insertion – swap) and order 6 (reverse – swap – insertion).

For this test, 180 instances, i.e. 5 instances for each of 36 combinations of 3 levels for the number of jobs (40, 60 and 80) and 3 levels for the number of stages (5, 10 and 15), were generated randomly using a real data of a Korean company. The operation and rework setup times were generated from $DU(5, 100)$ and $DU(5, 20)$, respectively, where $DU(l, u)$ denotes the discrete uniform distribution with range $[l, u]$. Also, the number of queue time limits for each job was set to 2, where the limits were generated from $DU(5, 20)$ and the two ending stages

specifying each queue time limit were selected randomly in such a way that some in-between stages are overlapped. Finally, the algorithms were terminated when there was no improvement for 30 consecutive iterations, i.e. $T_{GVNS} = 30$. Because the optimal solutions could not be obtained in reasonable time, the relative performance ratio (RPR) was used as a performance measure, where the RPR of an algorithm for an instance is calculated as

$$100 \cdot (M_a - M_{best}) / M_{best},$$

where M_a is the makespan obtained from the algorithm with combination *a* and M_{best} is the best one among those obtained from all combinations.

Test results are summarised in Figure 7 that shows the average RPR values for all 36 combinations. It can be seen from the figure that the best combination is order 3 (insertion – swap – reverse) in shaking and order 4 (insertion – reverse – swap) in local search improvement. To show the statistical difference among them, a Duncan's multiple range test was done and the result shows that the best combinations are order 3 – order 4, order 6 – order 1 and order 6 – order 1, which are denoted as GVNS₁, GVNS₂, and GVNS₃, respectively. See Table 3 for more details.

The first test was done to show the absolute performance of the VNS and the GVNS algorithms, together with the mixed integer programming model, for small sized instances. For the test, 60 instances were generated randomly, i.e. 10 instances for each of 6 combinations of 2 levels for the number of jobs (5 and 7) and 3 levels for the number of stages (5, 10, and 15). The data, including queue time limits, were generated by the same method used in the preliminary test. The algorithms were terminated when there is no improvement for 30 consecutive iterations, i.e. $T_{VNS} = T_{GVNS} = 30$. The performance measures used are: (a) percentage gaps from the optimal solution values; and (b) CPU seconds, where the percentage gap of an algorithm for an instance is calculated as

$$100 \cdot (M_a - M_{opt}) / M_{opt},$$

where M_a is the makespan obtained from algorithm *a* and M_{opt} is the optimal makespan obtained from CPLEX 20.1 with the time limit of 3600 s.

Test results are summarised in Table 4(a) that shows the number of instances that the CPLEX gave optimal solutions within 1 h and average percentage gaps of the VNS and the GVNS algorithms. It can be seen from the table that the CPLEX could not always give the optimal solutions for the small-sized test instances with 7 jobs. Also, the GVNS algorithms not only outperformed the VNS algorithm in overall average due to the enlarged

Table 4. Results for small sized test instances.

		CPLEX					
Number of jobs	Number of stages	N _{opt} ^a	NEH	VNS	GVNS ₁	GVNS ₂	GVNS ₃
(a) Percentage gaps from the optimal solution values							
5	5	10	9.10*	1.94	1.88	2.06	2.17
	10	10	12.02	2.35	2.17	2.56	2.47
	15	10	15.16	3.17	2.79	2.87	2.74
7	5	7	16.11	2.95	2.24	2.58	2.41
	10	6	18.69	3.18	2.65	2.88	2.68
	15	1	19.06	3.73	2.91	2.98	2.97
	Average		15.02	2.88	2.44	2.65	2.57
Number of jobs	Number of stages	CPLEX	VNS	GVNS ₁	GVNS ₂	GVNS ₃	
(b) CPU seconds.							
5	5	18.8*	4.3	3.6	5.4	4.3	
	10	105.5	11.3	11.6	12.2	12.0	
	15	386.7	18.5	17.3	21.1	17.7	
7	5	1675.6	6.5	8.1	7.4	7.3	
	10	3120.8	16.2	14.9	15.2	14.6	
	15	3512.5	21.9	23.8	21.3	25.0	

^aNumber of instances that CPLEX gave the optimal solutions within 3600 s.

*Average percentage gap from the optimal solution values for the instances that the CPLEX gave the optimal solutions.

*Average CPU second out of 10 instances.

Table 5. Results of paired *t*-tests for small sized test instances.

Algorithms (APG [†])	GVNS ₁	GVNS ₃	GVNS ₂
GVNS ₁ (2.44)			
GVNS ₃ (2.57)	**		
GVNS ₂ (2.65)	**	**	
VNS (2.88)	**	**	**

[†] Average percentage gap.

** Statistically different at a significant level of 0.05.

search space but also gave near optimal solutions for small-sized test instances within a reasonable amount of computation times. Among them, GVNS₁ performed the best, i.e. 2.44% in overall average. To test statistical difference among the algorithms, paired *t*-tests were done using the average percentage gaps for all test instances and the results are summarised in Table 5 that shows GVNS₁ outperforms the others statistically. Also, all the algorithms gave the solutions within 30 s for all the test instances. See Table 4(b) for the detailed CPU seconds.

The second test was done to show the relative performance of the VNS and the GVNS algorithms for medium-to-large-sized test instances. For the test, we generated 180 instances randomly, i.e. 10 instances for each of 18 combinations of 6 levels for the number of jobs (10, 20, 40, 60, 80 and 100) and 3 levels for the number of stages (5, 10 and 15). The data were generated using the same method. Test results are summarised in Table 6(a) that shows the RPR values of the algorithms. As in the results for small-sized test instances, GVNS₁ outperforms the others in overall average. In fact, the overall RPR value was 0.89% and the overall percentage improvement over the initial solutions was 6.87%,

which shows the effectiveness of the GVNS approach. Also, the results of the paired *t*-tests showed that GVNS₁ performed the best. See Table 7 for more details. Finally, Table 6(b) summarised the CPU seconds, which shows that there is not much difference due to the same termination condition.

5. Concluding remarks

This study addressed a new multi-stage flow shop scheduling problem in which a job is reworked if the queue time of the job between two stages exceeds an upper limit. Unlike the previous studies, the generalised type of overlapped queue time limits, i.e. in-between stages in queue time limits for a job may be overlapped, were considered. The problem is to determine the start times of jobs on each stage and rework setups/operations on the rework setup and corresponding stages if incurred for the objective of minimising makespan. After a mixed integer programming model was developed to represent the problem mathematically, a VNS algorithm was proposed that generates an initial solution by the NEH-based heuristic and improves it using a shaking and a local search improvement methods. Then, it was extended into the GVNS algorithms with VND algorithm in shaking and local search improvement. Computational results showed that the GVNS algorithms outperform the VNS algorithm significantly. Also, GVNS₁ with insertion/swap/reverse order for shaking and insertion/reverse/swap order for local search improvement performed the best and gave near optimal

Table 6. Results for medium-to-large sized test instances.

Number of jobs	Number of stages	VNS	GVNS ₁	GVNS ₂	GVNS ₃
(a) Relative performance ratios.					
10	5	2.56*	2.01	2.35	2.90
	10	2.60	0.49	2.59	2.85
	15	3.30	0.45	2.62	2.23
20	5	2.65	1.68	1.79	3.55
	10	2.63	0.33	3.83	2.61
	15	3.20	1.32	3.05	1.06
40	5	2.45	0.62	2.45	1.58
	10	2.51	0.36	2.63	0.82
	15	2.20	1.08	1.95	1.13
60	5	2.88	0.87	3.21	3.16
	10	3.92	1.61	3.03	2.09
	15	3.22	0.86	3.11	2.46
80	5	2.37	0.67	1.87	1.62
	10	1.46	0.40	2.18	1.42
	15	1.97	0.82	1.63	1.42
100	5	2.66	0.98	1.80	1.60
	10	1.17	0.52	1.06	1.21
	15	2.26	1.03	3.55	3.33
Average		2.55	0.89	2.48	2.05
Number of jobs	Number of stages	VNS	GVNS ₁	GVNS ₂	GVNS ₃
(b) CPU seconds					
10	5	8.5*	9.1	9.3	9.7
	10	20.9	24.2	21.5	20.3
	15	32.9	40.8	36.0	33.2
20	5	23.9	28.9	22.9	22.9
	10	41.8	50.0	36.5	59.2
	15	70.0	81.3	57.3	73.1
40	5	71.7	82.4	84.5	91.5
	10	139.4	135.4	123.7	169.5
	15	179.5	203.6	216.3	204.7
60	5	226.0	215.9	203.0	200.5
	10	304.6	273.7	323.9	370.8
	15	386.0	330.1	480.2	395.8
80	5	391.7	415.7	428.0	388.8
	10	564.4	514.8	598.5	729.1
	15	626.0	698.4	699.0	697.0
100	5	650.8	657.3	724.0	730.2
	10	1019.3	1187.7	1087.6	1014.8
	15	1238.5	1311.4	1223.1	1232.5

* Average relative performance ratio out of 10 instances.

* Average CPU seconds out of 10 instances.

Table 7. Results of paired t-tests for medium-to-large sized test instances.

Algorithms (ARPR [†])	GVNS ₁	GVNS ₃	GVNS ₂
GVNS ₁ (0.89)			
GVNS ₃ (2.05)	**		
GVNS ₂ (2.48)	**	**	
VNS (2.55)	**	**	=

[†] Average relative performance ratio.

** Statistically different at a significant level of 0.05.

= Statistically indifferent at a significant level of 0.05.

solutions for small-sized test instances, i.e. 2.44% optimality gap in overall average.

As a beginning study on multi-stage flow shop scheduling with reworks under overlapped queue time limits, this study can be extended in several ways. First, other state-of-the-art meta-heuristics can be developed to improve the solution quality. Second, other considerations, such as multi-objectives, sequence-dependent

setups and finite buffers, are worth to be considered. Finally, the current study can be extended to dynamic and stochastic ones with non-zero ready times, machine breakdowns, uncertain processing times, and so on.

Acknowledgment

This paper is an improved version of the first author's master dissertation (Kim, 2022).

Disclosure statement

No potential conflict of interest was reported by the author(s).

Funding

This work was supported by the National Research Foundation of Korea (NRF) grant funded by Korea government (MSIT). (No. 2022R1F1A1066641, Title: Hybrid Flow

Shop Scheduling with Reworks under Overlapped Queue Time Limits: Mathematical Models and Predictive/Reactive Algorithms)

Notes on contributors



Prof. Dong-Ho Lee is a professor at the Department of Industrial Engineering, Hanyang University, Seoul, Republic of Korea. He obtained his B.S. degree from Seoul National University and M.S./Ph.D. degrees from Korea Advanced Institute of Science and Technology (KAIST), all in Industrial Engineering. After earning his Ph.D. degree, he worked as a post-doctoral research fellow at the Department of Mechanical Engineering, Ecole Polytechnique Federale de Lausanne (EPFL), Switzerland. His research interests include design and operation of manufacturing/service systems, environmental conscious manufacturing, reverse logistics and operations research applications.



Mr. Hyeon-Il Kim is a Ph.D. student at the Department of Industrial Engineering, Hanyang University, Seoul, Republic of Korea. He obtained his B.S. degree from Kyungsoo University and M.S. degree from Hanyang University, all in Industrial Engineering. His research interests include production planning and scheduling in manufacturing systems and combinatorial optimisation.

Data availability statement

The data that support the findings of this study are available from the corresponding author, [Dong-Ho Lee], upon reasonable request.

ORCID

Dong-Ho Lee  <http://orcid.org/0000-0002-7679-5167>

References

- Akkerman, R., D. P. van Donk, and G. Gaalman. 2007. "Influence of Capacity-and Time-Constrained Intermediate Storage in Two-Stage Food Production Systems." *International Journal of Production Research* 45 (13): 2955–2973.
- An, Y.-J., Y.-D. Kim, and S.-W. Choi. 2016. "Minimizing Makespan in a Two-Machine Flowshop with a Limited Waiting Time Constraint and Sequence-dependent Setup Times." *Computers & Operations Research* 71: 127–136.
- Bai, X., and S. B. Gershwin. 1990. "A Manufacturing Scheduler's Perspective on Semiconductor Fabrication." In *VLSI Memo*. Massachusetts Institute of Technology, Cambridge Microsystems Research Center, 89–518.
- Benkalai, I., D. Rebaine, C. Gagné, and P. Baptiste. 2017. "Improving the Migrating Birds Optimization Metaheuristic for the Permutation Flow Shop with Sequence-dependent Set-up Times." *International Journal of Production Research* 55 (20): 6145–6157.
- Bouquard, J. L., and C. Lenté. 2006. "Two-machine Flow Shop Scheduling Problems with Minimal and Maximal Delays." *4OR* 4 (1): 15–28.
- Chen, J. S., and J. S. Yang. 2006. "Model Formulations for the Machine Scheduling Problem with Limited Waiting Time Constraints." *Journal of Information and Optimization Sciences* 27 (1): 225–240.
- Cho, Y., S.-K. Yoo, B.-H. Kim, and D.-H. Lee. 2019. "Multi-stage Hybrid Flowshop Scheduling with Reworks Under Queue Time Limits: Minimizing Makespan." In *Proceedings of the Spring KIIIE/KORMS/KSS Joint Conference*, Gwangju, Republic of Korea.
- Dhouib, E., J. Teghem, and T. Loukil. 2013. "Lexicographic Optimization of a Permutation Flow Shop Scheduling Problem with Time Lag Constraints." *International Transactions in Operational Research* 20 (2): 213–232.
- Gao, F., M. Liu, J. J. Wang, and Y. Y. Lu. 2018. "No-wait Two-machine Permutation Flow Shop Scheduling Problem with Learning Effect, Common Due Date and Controllable Job Processing Times." *International Journal of Production Research* 56 (6): 2361–2369.
- Gicquel, C., L. Hege, M. Minoux, and W. van Canneyt. 2012. "A Discrete Time Exact Solution Approach for a Complex Hybrid Flow-Shop Scheduling Problem with Limited-Wait Constraints." *Computers & Operations Research* 39 (3): 629–636.
- Ham, M. 2012. "Integer Programming-based Real-Time Dispatching (i-RTD) Heuristic for Wet-Etch Station at Wafer Fabrication." *International Journal of Production Research* 50 (10): 2809–2822.
- Ham, M., Y.-H. Lee, and J.-H. An. 2011. "IP-based Real Time Dispatching for Two Machine Batching Problem with Time Window Constraints." *IEEE Transactions on Automation Science and Engineering* 8 (3): 589–597.
- Hamdi, I., and T. Loukil. 2015. "Minimizing Total Tardiness in the Permutation Flowshop Scheduling Problem with Minimal and Maximal Time Lags." *Operational Research* 15 (1): 95–114.
- Hansen, P., and N. Mladenović. 2001. "Variable Neighborhood Search: Principles and Applications." *European Journal of Operational Research* 130 (3): 449–467.
- Hansen, P., and N. Mladenović. 2003. "Variable Neighborhood Search." In *Handbook of Metaheuristics*, 145–184. Boston, MA: Springer.
- Hansen, P., N. Mladenović, R. Todosijević, and S. Hanafi. 2017. "Variable Neighborhood Search: Basics and Variants." *EURO Journal on Computational Optimization* 5 (3): 423–454.
- Hansen, P., N. Mladenović, and D. Urošević. 2006. "Variable Neighborhood Search and Local Branching." *Computers & Operations Research* 33 (10): 3034–3045.
- Harjunkoski, I., and I. E. Grossmann. 2001. "A Decomposition Approach for the Scheduling of a Steel Plant Production." *Computers & Chemical Engineering* 25 (11-12): 1647–1660.
- Ho, M., F. Hnaien, and F. Dugardin. 2021. "Electricity Cost Minimisation for Optimal Makespan Solution in Flow Shop Scheduling Under Time-of-Use Tariffs." *International Journal of Production Research* 59 (4): 1041–1067.
- Jiang, E. D., and L. Wang. 2019. "An Improved Multi-Objective Evolutionary Algorithm Based on Decomposition for Energy-Efficient Permutation Flow Shop Scheduling

- Problem with Sequence-dependent Setup Time.” *International Journal of Production Research* 57 (6): 1756–1771.
- Jomaa, W., M. Eddaly, and B. Jarboui. 2021. “Variable Neighborhood Search Algorithms for the Permutation Flowshop Scheduling Problem with the Preventive Maintenance.” *Operational Research* 21 (4): 2525–2542.
- Joo, B.-J., and Y.-D. Kim. 2009. “A Branch-and-Bound Algorithm for a Two-machine Flowshop Scheduling Problem with Limited Waiting Time Constraints.” *Journal of the Operational Research Society* 60 (4): 572–582.
- Kim, H.-I. 2022. “Scheduling Algorithms for Multi-Stage Flow Shops with Reworks under Overlapped Queue Time Limits.” Master Dissertation, Hanyang University, Seoul, Republic of Korea.
- Kim, H.-J., and J.-H. Lee. 2019. “Three-machine Flow Shop Scheduling with Overlapping Waiting Time Constraints.” *Computers & Operations Research* 101: 93–102.
- Klemmt, A., and L. Mönch. 2012. “Scheduling Jobs with Time Constraints between Consecutive Process Steps in Semiconductor Manufacturing.” In *Proceedings of the 2012 Winter Simulation Conference*, Berlin, Germany, 2173–2182.
- Komaki, G. M., S. Sheikh, and B. Malakooti. 2019. “Flow Shop Scheduling Problems with Assembly Operations: A Review and New Trends.” *International Journal of Production Research* 57 (10): 2926–2955.
- Lee, J.-Y. 2020. “A Genetic Algorithm for a Two-machine Flowshop with a Limited Waiting Time Constraint and Sequence-dependent Setup Times.” *Mathematical Problems in Engineering* 2020: 8833645.
- Lee, J.-H., T.-S. Yu, and K. Park. 2021. “Scheduling of Flow Shop with Overlapping Waiting Time Constraints Using Genetic Algorithm.” *Journal of the Korean Institute of Industrial Engineers* 47 (1): 34–44.
- Li, Y., and Z. Dai. 2020. “A Two-stage Flow-Shop Scheduling Problem with Incompatible Job Families and Limited Waiting Time.” *Engineering Optimization* 52 (3): 484–506.
- M’hallah, R. 2014. “An Iterated Local Search Variable Neighborhood Descent Hybrid Heuristic for the Total Earliness Tardiness Permutation Flow Shop.” *International Journal of Production Research* 52 (13): 3802–3819.
- Mladenović, N., and P. Hansen. 1997. “Variable Neighborhood Search.” *Computers & Operations Research* 24 (11): 1097–1100.
- Moslehi, G., and D. Khorasani. 2014. “A Hybrid Variable Neighborhood Search Algorithm for Solving the Limited-Buffer Permutation Flow Shop Scheduling Problem with the Makespan Criterion.” *Computers & Operations Research* 52: 260–268.
- Nawaz, M., E. E. Enscore Jr, and I. Ham. 1983. “A Heuristic Algorithm for the m-Machine, n-Job Flow-Shop Sequencing Problem.” *Omega* 11 (1): 91–95.
- Panwalker, S. S., and C. Koulamas. 2021. “New Results for Minimising Variation of Flow Time in Two-machine Proportionate No-wait Flow Shops.” *International Journal of Production Research* 59 (9): 2789–2799.
- Pinedo, M. 2010. *Scheduling: Theory, Algorithms and Systems*. New York: Springer.
- Pirovano, G., F. Ciccullo, M. Pero, and T. Rossi. 2020. “Scheduling Batches with Time Constraints in Wafer Fabrication.” *International Journal of Operational Research* 37 (1): 1–31.
- Ramezani, R., M. M. Vali-Siar, and M. Jalalian. 2019. “Green Permutation Flowshop Scheduling Problem with Sequence-Dependent Setup Times: A Case Study.” *International Journal of Production Research* 57 (10): 3311–3333.
- Rossit, D. A., F. Tohmé, and M. Frutos. 2018. “The Non-Permutation Flow-Shop Scheduling Problem: A Literature Review.” *Omega* 77: 143–153.
- Samarghandi, H., and T. Y. ElMekkawy. 2012. “A Meta-Heuristic Approach for Solving No-wait Flow-Shop Problem.” *International Journal of Production Research* 50 (24): 7313–7326.
- Schaller, J., and J. M. Valente. 2021. “Scheduling in a No-wait Flow Shop to Minimise Total Earliness and Tardiness with Additional Idle Time Allowed.” *International Journal of Production Research*, doi:10.1080/00207543.2021.1962017.
- Scholl, W., and J. Domschke. 2000. “Implementation of Modeling and Simulation in Semiconductor Wafer Fabrication with Time Constraints between Wet Etch and Furnace Operations.” *IEEE Transactions on Semiconductor Manufacturing* 13 (3): 273–277.
- Shao, W., Z. Shao, and D. Pi. 2022. “Multi-Local Search-based General Variable Neighborhood Search for Distributed Flow Shop Scheduling in Heterogeneous Multi-Factories.” *Applied Soft Computing* 125: 109138.
- Shi, C., and S. B. Gershwin. 2016. “Part Sojourn Time Distribution in a Two-machine Line.” *European Journal of Operational Research* 248 (1): 146–158.
- Su, L. H. 2003. “A Hybrid Two-stage Flowshop with Limited Waiting Time Constraints.” *Computers & Industrial Engineering* 44 (3): 409–424.
- Wang, B., K. Huang, and T. Li. 2018. “Permutation Flowshop Scheduling with Time Lag Constraints and Makespan Criterion.” *Computers & Industrial Engineering* 120: 1–14.
- Wang, F. F. Ju, and N. Kang. 2019. “Transient Analysis and Real-Time Control of Geometric Serial Lines with Residence Time Constraints.” *IIE Transaction* 51 (7): 709–728.
- Wang, B., and T. Li. 2011. “A Comparison of Permutation Flowshop Scheduling Heuristics for Limited Waiting Time Constraints.” *Proceedings of the 2011 International Conference on Business Management and Electronic Information* 308–311.
- Wang, B. L., T. K. Li, and H. B. Ge. 2013. “Neighborhood Search Heuristic for 2-machine Flowshop Scheduling Problem with Limited Waiting Times.” *Applied Mechanics and Materials*, 411–414, 1894–1897.
- Wang, R., Y. Shen, W. Wang, and L. Shi. 2020. “Formulation and Methods for a Class of Two-stage Flow-Shop Scheduling Problem with the Batch Processor.” In *Proceedings of the 2020 IEEE International Conference on Automation Science and Engineering*, Virtual, 728–733.
- Wang, J. J., and L. Wang. 2019. “Decoding Methods for the Flow Shop Scheduling with Peak Consumption Constraints.” *International Journal of Production Research* 57 (10): 3200–3218.
- Wu, X., and A. Che. 2020. “Energy-Efficient No-wait Permutation Flow Shop Scheduling by Adaptive Multi-Objective Variable Neighborhood Search.” *Omega* 94: 102117.
- Xie, Z., C. Zhang, X. Shao, W. Lin, and H. Zhu. 2014. “An Effective Hybrid Teaching-Learning-Based Optimization Algorithm for Permutation Flow Shop Scheduling Problem.” *Advances in Engineering Software* 30 (2): 155–165.

- Xin, X., Q. Jiang, C. Li, S. Li, and K. Chen. 2021. "Permutation Flow Shop Energy-Efficient Scheduling with a Position-based Learning Effect." *International Journal of Production Research*, doi:10.1080/00207543.2021.2008041.
- Yang, D. L., and M. S. Chern. 1995. "A Two-machine Flow-shop Sequencing Problem with Limited Waiting Time Constraints." *Computers & Industrial Engineering* 28 (1): 63–70.
- Yenisey, M. M., and B. Yagmahan. 2014. "Multi-Objective Permutation Flow Shop Scheduling Problem: Literature Review, Classification and Current Trends." *Omega* 45: 119–135.
- Yu, T.-S., H.-J. Kim, and T.-E. Lee. 2017. "Minimization of Waiting Time Variation in a Generalized Two-machine Flow Shop with Waiting Time Constraints and Skipping Jobs." *IEEE Transactions on Semiconductor Manufacturing* 30 (2): 155–165.
- Zhao, F., X. Xu, L. Wang, T. Xu, N. Zhu, and Jonrinaldi. 2022. "A Reinforcement Learning-Driven Brain Storm Optimisation Algorithm for Multi-Objective Energy-Efficient Distributed Assembly No-wait Flow Shop Scheduling Problem." *International Journal of Production Research*, doi:10.1080/00207543.2022.2070786.
- Zheng, X., S. Zhou, R. Xu, and H. Chen. 2020. "Energy-Efficient Scheduling for Multi-Objective Two-stage Flow Shop Using a Hybrid Ant Colony Optimisation Algorithm." *International Journal of Production Research* 58 (13): 4103–4120.
- Zhou, Y., and K. Wu. 2017. "Heuristic Simulated Annealing Approach for Diffusion Scheduling in a Semiconductor Fab." In *Proceedings of IEEE/ACIS International Conference on Computer and Information Science*, Beijing, China, 785–789.